

Resolução ESTRUTURA DE DADOS

2018-2 PROVA 1

1) Abaixo está uma implementação de uma FilaSimples, encadeada. Preciso USAR essa implementação para criar uma FILA COM PRIORIDADES usando um conceito próximo a um HASH. Uma FILA COM PRIORIDADES é uma estrutura de dados onde há uma ordenação dos elementos no momento da inserção. A posição de inserção é definida pela prioridade do elemento. Cada elemento que entrar na estrutura trará consigo uma informação adicional que é a sua prioridade (ou seja, o dado e a prioridade). A prioridade é um valor que vai de 1 a 10, sendo 1 a maior prioridade e 10 a menor. As operações da FILA COM PRIORIDADES são EntrarFP(com 2 parâmetros) e SairFP (sem parâmetros). Na operação “SairFP”, quem deve sair é o elemento de maior prioridade dentre todos os presentes (caso haja mais de 1 com a mesma prioridade, deve sair o que está aguardando a mais tempo, ou seja, quem entrou primeiro).

a) Qual será a abordagem que você utilizará para fazer a alteração no código? Explique a idéia do que você vai implementar a seguir (1 pt)

b) Implemente a classe FILAPRIORIDADE com as 2 operações (e atributos) e faça as alterações necessárias no código da FilaSimples abaixo para atender o que é necessário. (2,5 pts)

<pre>class FilaSimples { Elemento prim; Elemento ult; /* construtor */ FilaSimples(){ prim=null; ult=null; } }</pre>	<pre>void entrarNaFila(int dado){ Elemento novo = new Elemento (dado); ult.setProx(novo); ult = novo; }</pre>	<pre>int sairDaFila() { Elemento aux = prim; prim = prim.getProx(); return aux.getDado(); } }</pre>
--	---	---

a) igual 2018-1

```
1 class No:
2     def __init__(self, dado, prioridade):
3         """
4         Inicializa um nó com dado e prioridade.
5         """
6         self.dado = dado
7         self.prioridade = prioridade
8         self.prox = None
9
10 class FilaPrioridade:
11     def __init__(self):
12         """
13         Inicializa uma fila com prioridades.
14         """
15         self.prim = None # Aponta para o primeiro elemento da fila
16         self.ult = None  # Aponta para o último elemento da fila
17
```

```

10 class FilaPrioridade:
11     """
12     """
18     def entrarFP(self, dado, prioridade):
19         #Insere um elemento na fila com base na prioridade.
20         novo_no = No(dado, prioridade)
21
22         # Caso especial: a fila está vazia
23         if self.prim is None:
24             self.prim = novo_no
25             self.ult = novo_no
26             return
27
28         # Caso especial: o novo nó tem maior prioridade que o primeiro
29         if prioridade < self.prim.prioridade:
30             novo_no.prox = self.prim
31             self.prim = novo_no
32             return
33
34         # Percorrer a lista para encontrar a posição correta
35         atual = self.prim
36         while atual.prox is not None and atual.prox.prioridade <= prioridade:
37             atual = atual.prox
38
39         # Inserir o novo nó na posição correta
40         novo_no.prox = atual.prox
41         atual.prox = novo_no
42
43         # Caso especial: atualizar o último elemento
44         if novo_no.prox is None:
45             self.ult = novo_no
46
47     def sairFP(self):
48         #Remove e retorna o elemento de maior prioridade.
49         if self.prim is None:
50             raise Exception("Fila está vazia!")
51
52         # O primeiro elemento da fila é o de maior prioridade
53         elemento = self.prim
54         self.prim = self.prim.prox
55
56         # Caso especial: se a fila ficou vazia, atualizar 'ult'
57         if self.prim is None:
58             self.ult = None
59
60         return elemento.dado
61

```

2) Sobre PILHAS, FILAS e LISTAS (não confundir):

a) O esquema de FILA Circular para implementar uma fila FIFO é uma boa ideia? O mesmo esquema também serve quando vamos implementar a fila com alocação dinâmica? (1 pt)

R: Sim, o esquema de FILA CIRCULAR resolve o problema de desperdício de espaço em filas. Quando elementos são removidos de uma fila (FIFO), os índices iniciais ficam vazios e inutilizáveis. Na FILA CIRCULAR, ao atingir o final do array, o ponteiro de inserção volta ao início, reaproveitando o espaço.

b) Buscas em LISTAS não são muito eficientes. diga o por quê? (1 pt)

R: Buscas em listas não são eficientes porque você precisa percorrer os elementos sequencialmente até encontrar o desejado. Isso gera uma complexidade de $O(n)$ no pior caso, não sendo possível acessos diretos por índices

c) Se formos obrigados a usar uma lista sequencial, temos que nos esforçar para oferecer a melhor busca possível. Sendo assim, é vantajoso que os dados estejam ordenados? Por quê? (1 pt)

R: Sim, quando os dados estão ordenados, podemos usar a busca binária em vez de uma busca linear convencional. A busca binária divide a lista em duas partes a cada passo, reduzindo drasticamente o número de comparações necessárias. Sua complexidade é $O(\log n)$, em vez de $O(n)$ na busca linear.

d) Considere que temos uma pilha vazia e uma fila vazia. Sobre elas é realizada a série de operações abaixo. (a, b, c, d e são elementos). Após essas instruções, desenhe a pilha e a fila, com os elementos que ainda estariam dentro de cada uma. (1 pt)

push (1); push (2); push (3); pop(); push (4); push (5); push (1); entra(pop()); entra(pop()); entra(pop()); sai(); entra(pop());

R: push e pop (pilha) // entra e sai (fila)

Pilha:

push(1): [1]
push(2): [1, 2]
push(3): [1, 2, 3]
pop(): [1, 2] (3 removido)
push(4): [1, 2, 4]
push(5): [1, 2, 4, 5]
push(1): [1, 2, 4, 5, 1]

Fila:

entra(pop()): Fila = [1] (1 removido da Pilha)
entra(pop()): Fila = [1, 5] (5 removido da Pilha)
entra(pop()): Fila = [1, 5, 4] (4 removido da Pilha)
sai(): Fila = [5, 4] (1 removido)
entra(pop()): Fila = [5, 4, 2] (2 removido da Pilha)

Resultado final:

Pilha: [1]

Fila: [5, 4, 2]

3) O Hash é uma estrutura que permite buscas rápidas, desde que o implementador saiba usá-lo. Quais são as características que o implementador deve observar, para tomar suas decisões e obter os melhores resultados nas buscas com Hash? (1 pt)

R: O implementador deve considerar características como:

- escolher uma boa função hash que minimize ao máximo o mínimo de colisões, garantido que os dados sejam distribuídos de forma uniforme e que a chave resulte sempre no mesmo índice
- escolher o número adequado para a tabela, preferencialmente números primos para uma melhor dispersão de dados
- escolher uma boa estratégia para tratar as colisões, como encadeamento por exemplo
- escolher as chaves adequadas para o hash, como matrícula, cpf, e afins, para garantir que sejam únicos

4) Imagine um belíssimo HASH de fator de carga 5, com 1.000 elementos. O desenvolvedor – que ouviu falar de busca binária em array - está pensando se valeria a pena transferir esses dados para UM ARRAY ORDENADO. Discuta e justifique os aspectos de eficiência nas operações de inserção, remoção e busca nesse HASH e no ARRAY ORDENADO equivalente. Em seguida, comente se continuaria afirmando o mesmo (vale/não-vale a pena) se a quantidade de dados a ser suportada fosse de 10.000, justificando. (1,5 pts)

R: igual 2018-1

2001-1 PROVA 1

1) Qual a utilidade do uso de um esquema circular na implementação de uma fila? (max. 5 linhas) – 1 pt.

R: igual as outras

2) Considere que existe uma tabela de espalhamento de tamanho N e um conjunto de dados de tamanho M , já distribuídos na tabela. Num determinado momento, há uma modificação nos requisitos do sistema e o conjunto de dados passa a ter o dobro do tamanho original ($M*2$). Para manter o desempenho da tabela de hash em relação à recuperação de elementos, uma alternativa é dobrar também o tamanho da tabela de espalhamento ($N*2$). No caso de executar-se esta ação em tempo de execução (com os primeiros M dados já na tabela), seria necessário re-distribuir todos os elementos já inseridos ou não? Por quê? (máx 5 linhas) – 2 pts.

R: Sim, porque o cálculo do índice na tabela depende do tamanho N , e ao dobrar N , a função de hash aplicada aos dados resultará em índices diferentes. Sem isso os elementos podem acabar em posições erradas, tornando impossível acessá-los corretamente. Esse processo é conhecido como re-hashing.

3) Considere um conjunto de dados com 1023 elementos que deve ser armazenado em uma árvore binária ordenada. Qual o tamanho mínimo de um vetor para suportar tal árvore segundo o esquema de implementação visto em aula? E o tamanho máximo? Por quê? 1pt.

R: Tamanho mínimo: $2^{10} - 1 = 1023$. O vetor terá esse tamanho pois cada elemento da árvore irá ocupar uma posição no vetor, considerando que ela é completa. O tamanho máximo seria de $2^{1023} - 1$, o que é impraticável devido ao desperdício de memória. Nesse caso a árvore estaria degenerada e seria equiparável a uma lista e teria altura de 1023.

4) Considere uma massa de dados de tamanho M que está sendo armazenada em uma árvore com N níveis. Com a inclusão de mais M elementos (ou seja, dobrando a quantidade de elementos na árvore), qual é o menor incremento possível no número de níveis?

Explique. – 1 pt

Explicação:

- Em uma árvore binária completa, o número máximo de elementos é dado por:

$$2^N - 1$$

Onde N é o número de níveis.

altura = nível

Exemplo:

- Se uma árvore completa com $N = 3$ níveis contém $2^3 - 1 = 7$ elementos ($M = 7$):
 - Ao dobrar os elementos ($2M = 14$), o número de níveis necessário será $N' = 4$, pois:
$$2^4 - 1 = 15 \text{ (suporta até 15 elementos, o suficiente para 14).}$$
- Incremento: $N' - N = 4 - 3 = 1$.

R: O menor incremento no número de níveis ao dobrar o número de elementos em uma árvore binária é **1 nível**, desde que a árvore esteja completa ou quase completa antes da inclusão.

5) Na retirada de um elemento de uma árvore binária, existem 3 procedimentos básicos, conforme o número de filhos este elemento tem (0, 1 ou 2 filhos). No caso de 2 filhos, o procedimento básico supõe o uso do SUCESSOR IMEDIATO. Explique o conceito de SUCESSOR IMEDIATO. É possível usar o ANTECESSOR IMEDIATO nesta tarefa? Por quê? 1 pt.

R: O sucessor imediato de um nó em uma árvore binária é o menor valor na subárvore direita desse nó. O conceito de sucessor imediato em uma árvore binária de busca (ABB) refere-se ao menor valor que é maior do que um determinado nó. Sim, é possível usar o antecessor imediato no lugar do sucessor imediato.

O procedimento seria:

- Substituir o valor do nó removido pelo valor do antecessor imediato.
- Em seguida, remover o antecessor imediato da subárvore esquerda, garantindo que a árvore permaneça ordenada.

O antecessor imediato pode ser usado de maneira similar ao sucessor imediato para substituir o nó a ser removido. É uma escolha válida e depende da implementação ou preferência. Ambos garantem a manutenção da propriedade da árvore binária de busca, onde todos os elementos à esquerda são menores e à direita são maiores.

6) Qual estrutura de dados você utilizaria para implementar um programa que, dado um texto qualquer, apresenta como resultado uma tabela contendo todas as palavras e o número de vezes que cada uma delas aparece. Apresente um algoritmo básico (máx 10 linhas) deste programa, considerando que existe uma função LerProximaPalavra (para que eu saiba como vocês resolveriam o problema) e explique o porquê de sua escolha, apresentando argumentos relacionados à eficiência da estrutura escolhida nas operações necessárias para este programa. Exemplo de resultado deste programa: (3 pts)

PALAVRAS CONTAGEM	
Qual	1
Estrutura	2
Dados	2
De	8
Para	10

R: Para este problema, a estrutura de dados mais adequada seria um dicionário (ou tabela de hash). Utilizar um dicionário é eficiente porque permite acesso rápido, inserção e atualização em tempo constante $O(1)$ na maioria dos casos, o que é ideal para contar a frequência de palavras.

```
def contar_palavras(texto):
    contagem_palavras = {}
    while True:
        palavra = LerProximaPalavra(texto)
        if not palavra:
            break
        if palavra in contagem_palavras:
            contagem_palavras[palavra] += 1
        else:
            contagem_palavras[palavra] = 1
    return contagem_palavras
```

2022-1 PROVA 1

1) Em relação às frases abaixo, explique rapidamente as afirmações ou discorde, justificando.

- a) A fila do RU é um patrimônio imemorial da UFSC, mas ela não é uma fila FIFO. (0,5 pt)
- b) Uma pilha de pratos não é uma pilha LIFO. (0,5 pt)
- c) A implementação de esquema de fila circular em Array é útil para evitar movimentação de dados (0,5).

R: a) A fila do RU (Restaurante Universitário) deveria ser uma fila FIFO (First In, First Out), onde o primeiro a entrar na fila seria o primeiro a ser atendido. No entanto, na prática, podem ocorrer situações que rompem o comportamento FIFO, como pessoas furando a fila ou grupos se juntando ao meio da fila, o que descaracteriza a estrutura FIFO.

b) Uma pilha de pratos é o exemplo clássico de uma estrutura LIFO (Last In, First Out). O último prato colocado na pilha é sempre o primeiro a ser retirado. Este comportamento é exatamente o conceito de LIFO

c) Concordo. A implementação de uma fila circular em um array é útil para evitar a movimentação constante de dados, pois reutiliza o espaço liberado no início do array. Isso é eficiente e mantém as operações de inserção e remoção em tempo constante $O(1)$, evitando a necessidade de deslocar elementos.

2) Sabemos que listas encadeadas são estruturas intrinsecamente sequenciais - e isso gera um monte de limitações sobre o desempenho de operações importantes, como a busca.

a) Se não tivermos a informação de quantos elementos há em uma lista, como podemos descobrir e acessar o elemento central de uma lista de 1001 elementos? Descreva (com suas palavras) o algoritmo para tal e diga qual o desempenho geral do mesmo (quantos passos precisamos dar para encontrar e acessar o elemento central) (1,0 pt)

b) Se tivermos uma lista duplamente encadeada (como a do desenho abaixo), podemos melhorar o tempo de resposta para essa pergunta. Para isso, podemos ter 2 iteradores, um que parte do início em direção ao final e outro que parte do final em direção ao início e; alternativamente, damos passos em cada um dos iteradores. Quando ambos estiverem no mesmo elemento, este será o elemento central, certo? Quantos passos o algoritmo terá dado para chegar aí? Implemente o código deste procedimento em uma lista duplamente encadeada (use os nomes das classes, funções e variáveis, conforme exemplo abaixo) e que, ao final, informe qual é o elemento central e também quantos elementos há na lista. (2,5 pts)

```
class No:
def __init__(self, d):
self.dado = d
self.prox = None
self.ant = None
class lista_2mte:
def __init__(self):
```

```
self.inicio = None
self.fim = None
```

```
# exemplo de fç de
# inserção na frente
def insereFrente(self, novoDd):
    novoNo = No(novoDd)
    novoNo.prox = self.inicio
    if self.inicio is not None:
        self.inicio.ant = novoNo
    self.inicio = novoNo
```

R: Algoritmo:

1. **Percorrer a Lista:** Primeiramente, percorra a lista para contar o número total de elementos.
2. **Calcular a Posição Central:** Determine a posição central, que é $1001/2 = 500$.
3. **Acessar o Elemento Central:** Percorra a lista novamente até a posição 500 para acessar o elemento central.

Desempenho Geral:

- **Primeira Passagem:** $O(n)$ - Contagem dos elementos.
- **Segunda Passagem:** $O(n)$ - Acesso ao elemento central.
- **Total:** $O(n) + O(n) = O(n)$. Precisaremos de 1001 passos na primeira passagem e aproximadamente 501 passos na segunda, totalizando cerca de 1502 passos.

```
class No:
    def __init__(self, d):
        self.dado = d
        self.prox = None
        self.ant = None

class Lista2Mte:
    def __init__(self):
        self.inicio = None
        self.fim = None

    def insereFrente(self, novoDd):
        novoNo = No(novoDd)
        novoNo.prox = self.inicio
        if self.inicio is not None:
            self.inicio.ant = novoNo
        else:
            self.fim = novoNo
        self.inicio = novoNo

    def encontrarCentral(self):
        iterInicio = self.inicio
        iterFim = self.fim
        passos = 0

        while iterInicio != iterFim and iterInicio.prox != iterFim:
            iterInicio = iterInicio.prox
            iterFim = iterFim.ant
            passos += 1

        # Para listas com um número ímpar de elementos, iterInicio e iterFim
        # no mesmo nó. Para um número par, iterInicio.prox será igual a iterFim
        elementoCentral = iterInicio if iterInicio == iterFim else iterInicio.prox
        return elementoCentral.dado, passos * 2 + 1 # Passos * 2 + 1 para :

# Exemplo de uso:
lista = Lista2Mte()
for i in range(1001, 0, -1):
    lista.insereFrente(i)
elementoCentral, numPassos = lista.encontrarCentral()
print(f"Elemento Central: {elementoCentral}, Total de Passos: {numPassos}")
```

b)

Explicação:

- **Iteradores:** Dois iteradores movendo-se alternadamente garantem que o número de passos para encontrar o elemento central é aproximadamente a metade do número total de elementos (500 passos cada, 1000 ao todo).
- **Eficiência:** O uso de dois iteradores reduz o número de passos necessários para encontrar o elemento central em comparação com a abordagem de lista encadeada simples.

3) Um hash tradicional é composto por uma tabela de espalhamento em um array de listas (ou seja, o chamado de Encadeamento direto, figura da esquerda). Imagine que, em vez de usar um array de listas, alguém resolve implementar como um array de árvores binárias de busca (figura da direita). Faz sentido? Analise os aspectos que julgar importante. (2,5 pts.)

R: Vantagens

1. Eficiência na Busca:

- **Listas Encadeadas:** A busca, inserção e remoção em listas encadeadas têm um desempenho médio de $O(n)$ na pior das hipóteses, quando todas as chaves são mapeadas para o mesmo bucket.
- **BSTs:** Utilizando árvores binárias de busca balanceadas, o tempo de busca, inserção e remoção pode ser melhorado para $O(\log n)$. Isso pode ser mais eficiente se houver muitas colisões.

2. Melhor Distribuição e Redução de Colisões:

- **Listas Encadeadas:** Podem sofrer muito com colisões e, portanto, o desempenho piora quando a carga de trabalho aumenta.
- **BSTs:** A distribuição dos elementos dentro das árvores pode ser mais equilibrada, reduzindo os impactos das colisões.

Desvantagens

1. Complexidade de Implementação:

- **Listas Encadeadas:** São simples de implementar e gerenciar.
- **BSTs:** São mais complexas de implementar e manter, especialmente se quisermos garantir que as árvores permaneçam balanceadas (usando AVL trees ou Red-Black trees, por exemplo).

2. Sobrecarga de Manutenção:

- **Listas Encadeadas:** A manutenção é simples, com custo linear na pior das hipóteses.
- **BSTs:** Requerem operações adicionais para balanceamento, o que pode adicionar sobrecarga em termos de tempo e memória.

3. Uso de Memória:

- **Listas Encadeadas:** Utilizam memória de forma relativamente eficiente.
- **BSTs:** Podem requerer mais memória, especialmente em casos onde as árvores não são bem balanceadas ou se os nós requerem mais metadados (como ponteiros para nós pais e filhos).

Considerações Finais

- **Contexto de Aplicação:** A escolha entre listas encadeadas e BSTs em uma tabela hash depende do contexto de aplicação. Para aplicações onde as colisões são raras e o tamanho dos dados é moderado, listas encadeadas podem ser suficientes.

- **Volume de Dados e Performance:** Para aplicações com grande volume de dados e muitas colisões, BSTs podem oferecer melhorias de desempenho significativas, especialmente em termos de busca e remoção.
- **Requisitos de Manutenção:** Se a aplicação pode suportar a complexidade adicional de implementação e manutenção, o uso de BSTs pode ser justificado.

4) Comente, justifique, responda (0,5 pt cada item)

a) Retirada de um elemento de uma árvore binária de busca: para a retirada de um nó que tenha 2 filhos, qual é; ou melhor, quais são os procedimentos básicos para realizar essa tarefa? Por que essas são as formas recomendadas?

Procedimentos básicos:

1. **Encontrar o nó a ser removido.**
2. **Identificar o nó substituto:**
 - Substitua o valor do nó removido pelo **sucessor imediato** (menor elemento da subárvore direita) **ou** pelo **antecessor imediato** (maior elemento da subárvore esquerda).
3. **Remover o nó substituto:**
 - O nó sucessor ou antecessor escolhido será **sempre um nó folha ou com apenas 1 filho**, tornando sua remoção direta.

Por que essas formas são recomendadas?

- Mantêm a **propriedade de ordenação** da árvore binária de busca.
- Garantem que todos os valores na subárvore esquerda continuem menores e na subárvore direita, maiores.
- O uso do sucessor ou antecessor é eficiente e evita a necessidade de reestruturar toda a árvore.

b) Quais são e para quê servem os algoritmos clássicos de caminhadas em árvores binárias?

Os algoritmos clássicos de caminhadas em árvores binárias são:

1. **Pré-ordem (Pre-order):**
 - Ordem: **Raiz** → **Esquerda** → **Direita**
 - **Servem para:** Copiar a estrutura da árvore, calcular expressões prefixas ou gerar representações hierárquicas.
2. **In-ordem (In-order):**
 - Ordem: **Esquerda** → **Raiz** → **Direita**
 - **Servem para:** Obter os elementos em **ordem crescente** (em árvores binárias de busca).
3. **Pós-ordem (Post-order):**
 - Ordem: **Esquerda** → **Direita** → **Raiz**
 - **Servem para:** Apagar ou liberar nós em uma estrutura (exemplo: liberar memória) e avaliar expressões pós-fixas.

c) A busca em uma árvore binária é geralmente melhor que em uma lista ordenada? Por quê?

A busca em uma árvore binária de busca é geralmente mais eficiente do que em uma lista ordenada porque:

- **Árvore binária de busca:** No pior caso, o tempo de busca é $O(\log n)$ se a árvore estiver balanceada, devido à divisão em subárvores em cada passo.
- **Lista ordenada:** No pior caso, o tempo de busca é $O(n)$, pois é necessário percorrer sequencialmente os elementos da lista.

d) Uma árvore binária degenerada apresenta desempenho de busca pior que uma lista.

Uma árvore binária degenerada (onde cada nó tem apenas um filho) apresenta desempenho de busca pior do que uma lista, pois na prática se comporta como uma lista ligada, tornando a busca sequencial com complexidade $O(n)$.

e) O que define a distribuição dos elementos e a organização da árvore binária de busca (não AVL) são os algoritmos de inclusão e de exclusão de elementos? Como assim?

Sim, os algoritmos de inclusão e exclusão definem a distribuição dos elementos.

Justificativa:

- Em uma **árvore binária de busca não balanceada**, a ordem de inserção dos elementos influencia diretamente a estrutura da árvore:
 - Se os elementos forem inseridos em **ordem crescente ou decrescente**, a árvore tende a ficar **degenerada** (desequilibrada).
- A **exclusão** também afeta a organização, pois pode remover nós e gerar "buracos", obrigando a árvore a reorganizar elementos (usando sucessor ou antecessor).
- Como a árvore **não é AVL** (ou seja, não realiza balanceamento automático), a estrutura resultante depende unicamente da ordem e método de inserção/exclusão.

Resumo: A **organização** e a **eficiência** da árvore binária de busca não AVL são definidas pelos algoritmos usados para inclusão e exclusão, pois não há garantias de balanceamento automático.

(resp2) A distribuição dos elementos em uma árvore binária de busca é influenciada pelos algoritmos de inclusão e exclusão:

- **Algoritmos de inclusão:** A ordem de inserção dos elementos determina a estrutura da árvore. Inserções sem uma estratégia de balanceamento podem resultar em uma árvore degenerada.
- **Algoritmos de exclusão:** A remoção de elementos também afeta a estrutura, especialmente ao substituir um nó com dois filhos, podendo alterar a profundidade da árvore.

Portanto, a forma como os elementos são inseridos e removidos define a configuração e a eficiência da árvore binária de busca. Se os elementos são inseridos ou removidos de maneira balanceada, a árvore manterá um formato eficiente para operações de busca, inserção e remoção.

2024-1 PROVA 1

2) Sobre LISTAS, PILHAS e FILAS (não confundir):

a) Por que é recomendável usar um esquema de FILA Circular ao implementar uma fila FIFO? Também serve quando implementado com alocação dinâmica? (0,5 pt)

R:

- **Fila Circular em uma implementação com array fixo:**
 - É recomendável porque permite **reaproveitar o espaço vazio** no início do array após remoções, eliminando a necessidade de deslocar os elementos. Isso evita o desperdício de memória e mantém a complexidade de inserção e remoção em **$O(1)$** .
- **Fila Circular com alocação dinâmica:**
 - Neste caso, o esquema de fila circular **não é necessário**, pois a memória pode ser alocada dinamicamente conforme a necessidade, permitindo que a fila cresça ou diminua sem restrição de tamanho. A alocação dinâmica resolve o problema do espaço "desperdiçado".

b) Buscas em LISTAS não são muito eficientes (porque são buscas sequenciais, certo? - nada de busca binária aqui), então se formos obrigados a usar uma lista, como podemos oferecer a melhor busca possível? (0,5 pt)

Manter a lista em uma ordem específica (normalmente crescente ou decrescente) pode ajudar significativamente. Quando a lista está ordenada, você pode interromper a busca assim que encontrar um valor maior que o que você está procurando. Isso reduz o número de comparações necessárias, OU **o uso de um hash seria viável e muito eficiente** para melhorar a busca em uma lista, especialmente quando precisamos de buscas frequentes (?) Em uma **lista encadeada** ou sequencial, a busca tem complexidade $O(n)$, pois precisamos percorrer todos os elementos até encontrar o desejado. Em uma **tabela hash**, a busca tem complexidade **$O(1)$** no caso médio, pois o hash permite acessar diretamente a posição onde o elemento deve estar. (?)

c) Uma PILHA é uma estrutura que pode ser usada para 'inverter' a ordem dos dados (0,5 pt)

Sim, uma pilha segue o princípio LIFO (Last In, First Out), onde o último elemento inserido é o primeiro a ser removido. Isso permite que, ao inserir uma sequência de elementos em uma pilha e removê-los em seguida, a ordem original dos elementos seja invertida. Esse conceito é frequentemente utilizado em algoritmos que requerem a reversão de dados, como a inversão de strings ou a avaliação de expressões.

3) Um hash tradicional é composto por uma tabela de espalhamento em um array de listas (ou seja, o chamado de Encadeamento direto, figura da esquerda). Imagine que, em vez de usar um array de listas, alguém resolve implementar como um array de árvores binárias de busca (figura do meio) e outro pensa em fazer um array de arrays (figura da direita). Faz sentido? Analise os aspectos que julgar importante. (3,5 pts.)

R: **Hash com Array de Listas** (Encadeamento Direto):

- É a solução **mais simples** e eficiente quando o fator de carga é baixo e a função de hash distribui os elementos uniformemente, ou seja, fácil de implementar
- Não é eficiente com muitas colisões.

Hash com Array de Árvores Binárias de Busca:

- Faz sentido quando há **muitas colisões** (alto fator de carga) e a lista encadeada se tornaria muito grande.
- Oferece busca mais eficiente $O(\log k)$, mas com **maior custo de implementação** e uso de memória.
- Não é recomendado se a árvore não for balanceada.

Hash com Array de Arrays:

- É uma alternativa moderada entre listas e árvores.
- Oferece desempenho similar ao encadeamento direto, mas com menor sobrecarga de memória em relação à árvore.
- Ainda possui busca e remoção lineares $O(k)$, como no encadeamento direto.

Comparação Geral

Aspecto	Array de Listas	Array de Árvores BST	Array de Arrays
Inserção	$O(1)$	$O(\log k)$ (balanceado)	$O(1)$ (amortizado)
Busca	$O(k)$	$O(\log k)$ (balanceado)	$O(k)$
Remoção	$O(k)$	$O(\log k)$	$O(k)$
Ordenação	Não mantida	Mantida	Não mantida
Complexidade de Implementação	Simples	Complexa	Moderada
Uso de Memória	Baixo	Alto (ponteiros extras)	Moderado
Desempenho com alto fator de carga	Degrada para $O(k)$	Melhor que lista ($O(\log k)$)	Degrada para $O(k)$

4) Considere que não sabemos quantos elementos há em uma lista duplamente encadeada (como da figura 1, mas queremos descobrir e acessar o elemento central dessa lista (que tem, por exemplo, 1001 elementos). Podemos usar a seguinte ideia: ter 2 iteradores, um que começa no início e avança em direção ao final e outro que vai do final em direção ao início e vamos avançando cada um dos iteradores – um de cada vez. Quando ambos estiverem no mesmo elemento, este será o elemento central – concorda? Implemente o código deste procedimento (use os nomes das classes, funções e variáveis, conforme exemplo abaixo) que, ao final, informa qual é o elemento central (3,5 pt)

class No:	def __init__(self):	def insereNaFrente(self,
def __init__(self, d):	self.inicio = None	novoDd):
self.dado = d	self.fim = None	novoNo = No(novoDd)
self.prox = None		novoNo.prox = self.inicio
self.ant = None	# exemplo de função de	if self.inicio is not None:
	# inserção na frente	self.inicio.ant = novoNo
class lista_2mte:		self.inicio = novoNo

```

1  class No:
2      def __init__(self, d):
3          self.dado = d          # Dado armazenado no nó
4          self.prox = None       # Ponteiro para o próximo nó
5          self.ant = None        # Ponteiro para o nó anterior
6
7  class lista_2mte:
8      def __init__(self):
9          self.inicio = None     # Ponteiro para o início da lista
10         self.fim = None        # Ponteiro para o final da lista
11
12         # Inserção no início da lista
13         def insereNaFrente(self, novoDd):
14             novoNo = No(novoDd)
15             novoNo.prox = self.inicio
16             if self.inicio is not None:
17                 self.inicio.ant = novoNo
18             self.inicio = novoNo
19             if self.fim is None: # Se a lista estava vazia
20                 self.fim = novoNo
21
22         # Inserção no final da lista
23         def insereNoFim(self, novoDd):
24             novoNo = No(novoDd)
25             if self.fim is None:
26                 self.inicio = novoNo
27                 self.fim = novoNo
28             else:
29                 self.fim.prox = novoNo
30                 novoNo.ant = self.fim
31                 self.fim = novoNo
32

```

```

31         self.fim = novoNo
32
33     # Encontrar o elemento central usando dois iteradores
34     def encontrarElementoCentral(self):
35         if self.inicio is None:
36             print("A lista está vazia.")
37             return None
38
39         iterador_inicio = self.inicio # Iterador que parte do início
40         iterador_fim = self.fim       # Iterador que parte do final
41         passos = 0                    # Contador de passos
42
43         # Percorrer a lista até os dois iteradores se encontrarem
44         while iterador_inicio != iterador_fim and iterador_inicio.prox != iterador_fim:
45             iterador_inicio = iterador_inicio.prox
46             iterador_fim = iterador_fim.ant
47             passos += 1
48
49         # Encontramos o elemento central
50         print(f"Elemento central: {iterador_inicio.dado}")
51         print(f"Número de passos: {passos + 1}") # +1 porque o último passo conta o encontro
52         return iterador_inicio.dado
53

```

```

62
63     # Código de Teste
64     if __name__ == "__main__":
65         lista = lista_2mte()
66
67         # Inserindo 1001 elementos na lista (exemplo)
68         for i in range(1, 1002): # Adiciona os números de 1 a 1001
69             lista.inserirNoFim(i)
70
71         # Exibindo a lista (opcional)
72         print("Lista duplamente encadeada:")
73         lista.exibirLista()
74
75         # Encontrar o elemento central
76         lista.encontrarElementoCentral()
77

```